CSED233 · 데이터구조 · 포스텍

Ch2. One-Day C++ — 자료구조 구현 토대

1. 컴파일 파이프라인

이론 & 배경

- **C++**: C 확장, cross-platform, OOP(class), 고성능. 주요 버전 C++11/14/17/20/23.
- **Build 과정**:
  - **Compiler(컴파일러)**: `.cpp/.h` → `.o` (linux) / `.obj` (windows) (machine-code 변환)
  - **Linker(링커)**: 여러 `.o` + library → 실행 파일(`.exe` / linux 바이너리)
- 실습: `g++ -o my_main main.cpp` → `./my_main`

Q1. Compiler의 입출력은?

> **힌트**

컴파일은 소스 파일 단위로 일어나고, 결과물은 기계어 오브젝트 파일이다.

> **정답**

개념은 힌트 참고.

In: `.cpp` , `.h` / **Out**: `.o` , `.obj` (번역 단위 = .cpp 하나당 하나; .h는 #include로 포함)

Q2. Linker의 입출력은?

> **힌트**

링커는 여러 오브젝트 파일을 하나의 실행 파일로 묶는다.

> **정답**

개념은 힌트 참고.

In: 다수의 `.o` / **Out**: `.exe` (Windows) / 확장자 없는 실행 파일(Linux)

2. 변수 · 타입 · Namespace(네임스페이스)

기본 타입

기본 타입: C++의 기본 타입에는 bool, char, short, int, long, long long, float, double 등이 있다. 실제 크기와 표현 범위는 구현에 따라 달라질 수 있다.

- **Namespace** `std::` : `cout` , `cin` , `endl` 등 표준 라이브러리 식별자 scope.
- `using std::string;` 으로 prefix 생략 가능.

Float64 vs Float32

- 일반적인 IEEE 754 환경에서 float는 32비트 단정밀도, double은 64비트 배정밀도를 사용한다. C++ 표준이 모든 환경에서 이 비트 수를 강제하는 것은 아니다.
- double은 일반적으로 float보다 높은 정밀도를 제공하지만, 이진 부동소수점의 반올림 오차를 제거하지는 못한다.
- float는 GPU 연산이나 대규모 배열에서 메모리 사용량을 줄이는 데 유리할 수 있다. 실제 연산 속도상의 이점은 하드웨어와 연산 종류에 따라 달라진다.
- 정확한 금액 표현이 필요한 금융 계산에서는 최소 화폐 단위의 정수 표현 또는 적절한 십진 연산 방식을 고려해야 한다.

Q3. 직접 만든 `void cout(...)` 와 `std::cout` 을 동시에 사용하려면?

> **힌트**

전역 네임스페이스의 함수는 ::함수이름으로 명시할 수 있다. 반면 std::cout은 std 네임스페이스에 정의된 출력 스트림 객체로, 함수가 아니다. 두 이름은 ::와 std::로 구별할 수 있다.

> **정답**

개념은 힌트 참고.

사용자 정의 전역 함수는 ::cout(...)으로 호출하고, 표준 출력 스트림에는 std::cout << 값 형태로 값을 출력한다.

```
#include <iostream>
#include <string>

void cout(const std::string& s) {
```

```
    std::cout << "global: " << s << '\n';
}

int main() {
    ::cout("hello");          // 직접 만든 함수
    std::cout << "hello\n";   // 표준 출력 스트림
}
```

두 방식은 동일한 프로그램에서 함께 사용할 수 있다.

3. Pointer(포인터), Reference(레퍼런스), Array (DS 구현 핵심)

Pointer(포인터)

- 모든 변수는 메모리 주소를 가짐 → **pointer**는 그 주소를 저장하는 변수.
- 타입 `T*` = T 변수에 대한 포인터.
- 연산자:
  - &x는 주소를 취할 수 있는 객체 x의 주소를 구한다. *q는 포인터 q를 역참조하여 가리키는 객체에 접근한다. 이 연산을 안전하게 사용하려면 q가 현재 수명이 유효한 객체를 가리키고, 역참조할 수 있는 위치에 있어야 한다. nullptr, 해제된 객체를 가리키는 댕글링 포인터, 배열 끝 다음 위치를 가리키는 포인터 등을 역참조하면 정의되지 않은 동작이 발생한다. 연결 리스트 등의 자료구조에서는 노드의 삭제와 포인터 갱신 순서를 올바르게 유지해야 한다.

```
char ch = 'Q';
char* p = &ch;   // p holds address of ch
cout << *p;      // 'Q'
*p = 'X';        // ch 값이 'X'로 변경
```

Array(배열) ↔ Pointer(포인터) 동등성

- `T[N]` : 같은 타입 N개 연속 저장.
- **배열명 = 첫 원소 pointer**: 대부분의 식에서 그렇게 변환 (sizeof, 단항 &는 예외 — 배열 자체 취급)

```
char c[] = {'c','a','t'};
char* p = c;        // p → c[0]
char* q = &c[0];    // q → c[0]
cout << c[2] << p[2] << q[2]; // "ttt"
```

- 초기화 시 크기 생략 가능: `int a[] = {10,11,12,13};`

4. Dynamic Memory(동적 메모리) (new/delete)

Heap(힙) 할당

- 동적 할당은 프로그램 실행 중 필요한 객체의 저장 공간을 확보하고 객체의 수명을 관리하는 방법이다.
- new T는 객체를 위한 저장 공간을 확보하고 객체의 초기화를 수행한 뒤 T*를 반환한다. 클래스 타입이라면 적절한 생성자가 호출된다.
- new int처럼 초기화식을 생략하면 기본 타입의 값은 초기화되지 않을 수 있다. new int{}는 값을 0으로 초기화한다.
- new T로 생성한 객체는 소유권을 가진 코드에서 delete로, new T[n]으로 생성한 배열은 delete[]로 해제한다. 두 방식을 섞으면 정의되지 않은 동작(UB)이 발생한다.
- C++에는 일반적으로 사용되지 않는 객체를 자동 탐지해 회수하는 내장형 가비지 컬렉터가 없다. 소유한 동적 자원은 적절히 해제해야 한다.
- std::unique_ptr, std::vector와 같은 RAII 기반 객체에 소유권을 맡기면 수명 종료 시 자원이 자동 정리된다. 이러한 자원을 별도로 delete해서는 안 된다.
- 할당 자원의 소유권을 잃고 해제할 방법이 없어지면 메모리 누수(memory leak)가 발생할 수 있다.

배열 동적 할당

```
char* buffer = new char[500];
buffer[3] = 'a';
delete[] buffer;   // 배열은 delete[] 필수
```

- 핵심: 일반 객체는 `delete`, 배열은 `delete[]`. 섞으면 UB.

5. Reference (별칭)

참조

- `T&`: 기존 객체의 **alias**. 선언 시 반드시 초기화, NULL 불가.
- Pointer와 차이: reference는 재할당 불가, dereference 불필요.

```
string author = "Samuel Clemens";
string& penName = author;  // alias
penName = "Mark Twain";    // author도 바뀜
```

함수 인자 전달

```
void f(int value, int& ref) {
  value++;  // 원본 영향 없음 (pass by value)
  ref++;    // 원본 수정 (pass by reference)
}
```

- **DS 구현에서 중요**: 크기 큰 객체 복사 비용 회피, 원본 수정이 필요할 때.

6. 제어 흐름 & 함수

For loop

for (초기식; 조건식; 증감식) { 본문 }의 실행 순서는 다음과 같다. 초기식은 처음 한 번 실행된다. 이후 조건식을 평가하여 참이면 본문을 실행하고, 증감식을 실행한 뒤 다시 조건식을 평가한다. 조건이 거짓이면 반복을 종료한다. break는 가장 안쪽의 반복문 또는 switch를 즉시 종료한다. continue는 현재 반복의 남은 본문을 건너뛴다. for문에서는 continue 이후에도 증감식을 실행한 다음 조건식을 재평가한다.

```
#include <iostream>

int main() {
  for (int i = 0; i < 5; ++i) {
      if (i == 1) continue;
      if (i == 3) break;
      std::cout << i << ' ';
  }
  // 출력: 0 2
}
```

Function

- **Declaration** (prototype): 타입 + 이름 + 인자 리스트
- **Definition**: declaration + body

- Function Overloading(함수 오버로딩): 같은 범위에서 동일한 이름을 가진 함수를 매개변수 목록이 구별되도록 여러 개 선언하는 기능이다. 매개변수의 개수, 타입, 순서 등에 따라 서로 다른 함수가 선택될 수 있다. 반환형만 다른 함수 선언은 오버로딩으로 인정되지 않는다. 함수 호출 시 전달한 인자와 오버로드 후보 사이의 변환 및 일치 정도를 기준으로 호출할 함수를 결정하며, 적절한 함수를 하나로 결정할 수 없으면 컴파일 오류가 발생한다.

```
struct Passenger;

void print(int x);
void print(int x, int y);
void print(const Passenger& p);

// 아래 선언은 반환형만 다르므로 추가할 수 없다.
// double print(int x);
```

7. Struct(구조체) vs Class(클래스)

차이

- struct와 class는 멤버 변수, 멤버 함수, 생성자, 상속 등 대부분의 기능을 동일하게 지원한다.
- struct는 멤버의 기본 접근 권한이 public이고, 상속을 선언할 때 기본 상속 접근 권한도 public이다.
- class는 멤버의 기본 접근 권한이 private이고, 기본 상속 접근 권한도 private이다.
- 단순한 데이터 묶음에는 struct, 캡슐화된 추상 타입에는 class를 주로 사용하는 것은 코딩 관례이지 언어가 강제하는 기능적 차이는 아니다.

구분	struct	class
멤버 기본 접근 권한	public	private
기본 상속 접근 권한	public	private
생성자·멤버 함수	지원	지원
일반적인 사용 관례	데이터 묶음	캡슐화된 추상 타입

두 키워드는 기본 접근 권한에서 차이가 나지만, 명시적인 접근 지정자를 사용하면 같은 기능을 구현할 수 있다. 데이터 묶음과 추상 타입이라는 구분은 문법적 강제가 아닌 일반적인 코딩 관례다.

String (STL)

```
#include <string>
using std::string;

string s = "to be";
string t = "not" + s;       // "notto be"
string::size_type i = s.size(); // 5
char c = s[3];              // 'b'
s += " Smith";              // s == "to be Smith"
```

- Lexicographic comparison(사전식 비교): std::string은 기본 비교 시 문자 코드 단위를 왼쪽부터 차례로 비교한다. 처음으로 다른 위치의 코드 단위가 순서를 결정하며, 앞부분이 모두 같다면 더 짧은 문자열이 먼저 온다. 일반적인 ASCII 문자만 사용한다면 ASCII 코드 순서에 따라 비교된다. 하지만 이 방식은 로케일에 맞는 자연어 사전순이나 Unicode 정규화를 자동으로 제공하지 않는다.

8. Casting(캐스팅)

명시적 캐스팅

```
int cat = 14;
double dog = (double)cat;  // C-style
double pig = double(cat);  // functional
```

함정

```
int i1=18, i2=16;
double dv1 = i1/i2;                 // 1.0 (int 나눗셈 먼저!)
double dv2 = double(i1)/double(i2);  // 1.125
double dv3 = double(i1/i2);         // 1.0 (이미 int)
```

- **핵심**: 나눗셈 **전**에 캐스팅해야 함. 괄호 위치 주의.

보충: Header file · Namespace · I/O · Enumeration

- Header file(.h, .hpp): 여러 소스 파일에서 공유할 함수·타입 등의 선언을 담는다. #include 지시문으로 필요한 선언을 번역 단위에 포함시킨다.

- Namespace: 이름 충돌을 줄이기 위해 식별자를 구분하는 범위다. std::cout, std::cin은 표준 라이브러리의 이름을 명시적으로 사용한 예다.
- 표준 입출력: <iostream>을 포함한 뒤 std::cin >> 변수로 입력을 받고, std::cout << 값으로 출력한다. >>는 값을 읽어 변수에 저장하고, <<는 값을 출력 스트림으로 보낸다.
- Enumeration(열거형): 관련된 이름 붙은 정수 상수의 집합을 정의한다. enum class는 열거자 이름을 타입의 범위 안에 두며 다른 열거형이나 정수와의 암시적 혼용을 제한한다.
- Reference: T&는 기존 객체의 별칭이다. 함수 인자로 원본을 수정하거나 불필요한 복사를 피하는 데 사용한다.

```
#include <iostream>

enum class Direction {
    Left,
    Right
};

int main() {
    int n;
    std::cin >> n;

    Direction dir = Direction::Left;
    if (dir == Direction::Left) {
        std::cout << n << '\n';
    }
}
```

Summary

- Compiler/Linker
- Variable, pointer, array, dynamic memory, function, class/struct, string
- Header file, namespace, cin/cout, enumeration, reference
- Casting, Loop, break/continue